



UNIVERSIDAD CENTRAL DE VENEZUELA

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA ELÉCTRICA

COORDINACIÓN DE PASANTÍAS Y SERVICIO COMUNITARIO

RICARDO SANTANA

PASANTÍAS:

CONTROL DE UN MOTOR DE PASO MEDIANTE UN CONTROLADOR COMERCIAL

FI-UCV

2025

RICARDO SANTANA

PASANTÍAS:
CONTROL DE UN MOTOR DE PASO MEDIANTE UN
CONTROLADOR COMERCIAL

Presentado ante la Escuela de Ingeniería Eléctrica por el
Br. Ricardo Santana para aprobar el curso de pasantías.

Orientador: Prof. Ing. Alejandro Herrera

Pasantías:

Control de un motor de paso mediante un controlador comercial

Resumen

El proyecto consistió en el desarrollo integral de un sistema de control de movimiento en lazo cerrado basado en el microcontrolador ESP32. El sistema es capaz de comandar un motor de paso de alta potencia a través de un driver comercial, utilizando la lectura de posición de un encoder incremental como realimentación. La arquitectura de software, desarrollada sobre el framework ESP-IDF y el sistema operativo FreeRTOS, es modular y concurrente, permitiendo una operación robusta y en tiempo real. Para facilitar la interacción y el diagnóstico, se implementó una interfaz de usuario dual que incluye una pantalla LCD para la visualización de datos y una consola serie para la configuración de parámetros. El resultado final del proyecto no solo es un sistema funcional, sino también un conjunto completo de documentación técnica y software reutilizable.

Palabras-clave: Control. Motor. Encoder. Pantalla.

Lista de figuras

Figura 1 – Motor de paso	8
Figura 2 – Controlador digital de motor de paso	8
Figura 3 – Encoder rotativo	9
Figura 4 – ESP32-WROOM-32	11
Figura 5 – Pantalla LCD.	12
Figura 6 – Adaptador I2C.	13
Figura 7 – convertidor LM2596.	13
Figura 8 – Circuito de salida del encoder (basado en hoja de datos E6B2-CWZ6C). Los pines: marrón = Vcc (5–24 V), azul = GND, negro = fase A, blanco = fase B, naranja = fase Z.	14
Figura 9 – Señales esperadas en las fases A, B y Z (timing y desfase) por salida de colector abierto.	14
Figura 10 – Montaje de prueba del encoder: conexiones a la fuente de 5 V y sondas de osciloscopio en A y B.	15
Figura 11 – Señal observada en prueba: canales A y B en cuadratura.	15
Figura 12 – Montaje físico del encoder conectado al ESP32 durante las pruebas de integración.	16
Figura 13 – Captura de la salida serial o terminal del ESP32 mostrando conteo de pulsos, dirección y detección de Z.	16
Figura 14 – Diagrama de conexiones driver y motor, en conjunto con el ESP32. . .	17
Figura 15 – Diagrama de conexiones para utilizar motor.	19
Figura 16 – Diagrama de conexiones de sistema.	20
Figura 17 – Diagrama de conexiones de integración encoder y esp32.	21
Figura 18 – Diagrama de conexiones driver y motor, en conjunto con el esp32. . . .	23
Figura 19 – Pinout esp32-wroom-32.	41

Lista de tablas

Tabla 1 – Comparación de Motores Paso a Paso Unipolares y Bipolares	4
Tabla 2 – Características del encoder.	10
Tabla 3 – Tabla de Especificaciones (Specification)	40
Tabla 4 – Tipo de Conexiones (TYPE OF CONNECTION)	40

Índice General

Índice General	vi
1 –Introducción	1
2 –Objetivos	2
2.1 Objetivo General.	2
2.2 Objetivos Específicos.	2
3 –Marco Teórico	3
3.1 Fundamentos de Motores Paso a Paso (Stepper Motors) (LEILI MOTOR, 2022)	3
3.1.1 Tipos de Motores Paso a Paso	3
3.2 Principios Generales del Encoder (Codificador) (WEST INSTRUMENTS DE MÉXICO, S.A.,)	3
3.2.1 Tipos de Codificación	4
3.2.2 Tecnologías y Parámetros de Medición	5
4 –Marco Metodológico	6
5 –Trabajo Realizado	8
5.1 Componentes	8
5.1.1 Motor de paso 23HS22-2804S STEPPERONLINE	8
5.1.2 Digital Stepper Drive DM542T.	8
5.1.3 ROTATORY ENCODER (INCREMENTAL) E6B2-CWZ6C OM- RON Corporation.	9
5.1.4 ESP32-WROOM-32	11
5.1.5 Pantalla LCD WH1602B 16 caracteres x 2 líneas.	12
5.1.6 Adaptador I2C para pantalla LCD.	13
5.1.7 Convertidor DC DC Arduino basado en LM2596.	13
5.2 Implementación de Encoder	13
5.2.1 Caracterización y conexión del encoder	13
5.2.2 Montaje y señal observada	14
5.2.3 Integración con ESP32: lectura y salida de datos	15
5.3 Implementación del driver y motor	16
5.3.1 Conexiones y consideraciones de señal	16
5.3.2 Cálculo de pulsos por revolución y configuración de microstepping	17
5.3.3 Frecuencia de pulsos: mínimos y máximos	18

5.3.4	Generación de pulsos desde el ESP32	18
5.3.5	Prueba de una vuelta — procedimiento y observaciones	18
5.4	Diagrama de Conexiones.	19
6	–Resultados	21
6.1	Encoder	21
6.2	Motor	23
7	–Conclusión	27
	 Referencias	 28
	 Apêndices	 29
	 APÊNDICE A –Códigos	 30
A.1	Códigos libreria encoder	30
A.2	Códigos libreria stepperMotor	34
	 Anexos	 39
	 ANEXO A –Información sobre Motor de paso 23HS22-2804S	 40
	 ANEXO B –Información sobre el ESP32-WROOM-32	 41

1 Introducción

2 Objetivos

2.1 Objetivo General.

- Desarrollar una aplicación embebida que controle un motor de paso mediante un controlador comercial, en respuesta a la variación del ángulo de un encoder, mostrando los parámetros clave en una pantalla LCD mediante comunicación I2C o serial.

2.2 Objetivos Específicos.

- Estudiar los componentes seleccionados: microcontrolador, controlador del motor, motor de paso, encoder y pantalla LCD.
- Implementar la lectura del encoder y su conversión a señales de control para el motor de paso.
- Configurar la comunicación entre el microcontrolador y el controlador del motor.
- Desarrollar la interfaz de visualización en la pantalla LCD (I2C o serial).
- Integración, validación y documentación del sistema.

3 Marco Teórico

3.1 Fundamentos de Motores Paso a Paso (Stepper Motors) ([LEILI MOTOR, 2022](#))

Un motor paso a paso es un motor eléctrico que se caracteriza por su capacidad para girar en **pasos discretos** en lugar de hacerlo de manera continua, a diferencia de los motores convencionales. Este movimiento escalonado garantiza un **control preciso de la posición y la velocidad**, lo que los convierte en elementos esenciales para aplicaciones que exigen alta precisión, como la robótica, la automatización, las impresoras 3D y las máquinas CNC.

3.1.1 Tipos de Motores Paso a Paso

La clasificación principal se centra en los motores unipolares y bipolares. La diferencia fundamental reside en la configuración de sus devanados y la complejidad del control requerido:

- **Motor Paso a Paso Unipolar:** Cada fase posee un devanado con toma central, de modo que solo **la mitad de la bobina se energiza** en un momento dado. Esto resulta en un **par de salida menor** y una menor eficiencia. Sin embargo, su **control es más sencillo** y su funcionamiento tiende a ser más suave, con menor vibración y ruido. Son adecuados para aplicaciones de baja potencia.
- **Motor Paso a Paso Bipolar:** Utiliza un **único devanado por fase** que debe ser energizado en ambas direcciones. Al utilizar todo el devanado, generan un campo magnético más potente, lo que resulta en un **mayor par de salida** y una mayor eficiencia en términos de consumo de energía. Requieren circuitos de control más complejos para invertir la dirección de la corriente. Son ideales para aplicaciones de alto par, como máquinas CNC y robótica de alto rendimiento.

La siguiente tabla resume las diferencias clave de eficiencia y control:

3.2 Principios Generales del Encoder (Codificador) ([WEST INSTRUMENTS DE MÉXICO, S.A.,](#))

Los **Encoders** son **sensores que generan señales digitales en respuesta al movimiento**. Su función primaria es actuar como **transductores de retroalimen-**

Tabla 1 – Comparación de Motores Paso a Paso Unipolares y Bipolares

Característica	Unipolar	Bipolar	Fuente
Par de Salida	Menor	Mayor	
Eficiencia	Menor	Mayor	
Complejidad del Circuito	Más simple	Más complejo	
Vibración/Ruido	Menor/Más silencioso	Mayor/Más ruidoso	
Uso del Devanado	Mitad de la bobina	Bobina completa	

tación esenciales en el control de movimiento para medir velocidad, posición y como entrada para controles de rango. Existen tipos que responden a la rotación (rotatorios) y al movimiento lineal (lineales).

3.2.1 Tipos de Codificación

Los encoders se clasifican fundamentalmente según el tipo de datos que generan:

- **Encoder Incremental:** Genera **pulsos** mientras el dispositivo se mueve, proporcionando un número específico de pulsos por revolución (PPR). Se utiliza para medir la velocidad o la trayectoria de posición. Para obtener la posición, los pulsos deben ser acumulados por un contador, y la cuenta es susceptible a pérdida ante una interrupción de energía, requiriendo un reposicionamiento a una referencia u origen al iniciar.
 - **Cuadratura:** Cuando se requiere detectar el sentido de la dirección (movimiento bidireccional), se utiliza la salida de cuadratura, la cual consta de dos canales (A y B) desfasados 90° eléctricos.
 - **Índice (Canal Z):** Muchos encoders incrementales producen una señal adicional, el "índice" o "Canal Z", que se genera **una vez por revolución** y se usa para localizar una posición de origen específica.
- **Encoder Absoluto:** Genera un **mensaje digital multi-bit** que representa la posición actual del encoder. La ventaja clave es que, si se pierde la energía, su salida **se corrige inmediatamente** al restablecerse, eliminando la necesidad de volver a una posición de referencia. Su resolución se define por el número de bits por mensaje de salida.
 - **Código de Salida:** La salida puede ser en código binario o en **código Gray**, siendo este último preferido porque produce solamente un cambio de un solo bit en cada paso, lo que ayuda a reducir errores de lectura.
 - **Analogía:** La diferencia entre incremental y absoluto es análoga a la de un cronómetro (mide el tiempo transcurrido, incremental) frente a un reloj normal (indica constantemente la hora exacta o posición, absoluto).

3.2.2 Tecnologías y Parámetros de Medición

- **Tecnología Operacional:** Los encoders emplean principalmente tecnología **Óptica** (utiliza un LED brillando a través de un disco con patrones hacia fotodetectores, ofreciendo altas resoluciones y altas velocidades) y **Magnética** (más resistente al polvo, humedad, golpes térmicos y mecánicos, utiliza campos magnéticos o dispositivos de efecto Hall/magneto resistivos, y puede operar hasta una **velocidad de 0**).
- **Resolución y Precisión:** La **Resolución** es el número de segmentos de medición o unidades por revolución del eje. Es fundamental que la resolución sea igual o mejor a la requerida por la aplicación. La **Precisión**, sin embargo, es la exactitud con la que opera el transductor. La precisión del sistema se ve limitada por la **calidad del movimiento mecánico** de la máquina, como errores o contragolpe.
- **Ejemplo de Encoder Rotatorio (E6B2):** El modelo E6B2 es un encoder incremental de propósito general que soporta un amplio rango de voltaje operativo (**5 a 24 VDC** para el modelo de colector abierto). Presenta resoluciones de hasta **2,000 pulsos por revolución (P/R)** en un cuerpo de 40 mm, con salidas de fase A, B y Z. Un modelo disponible ofrece **salida de línea diferencial (*line driver*)** compatible con RS-422A, lo que permite la transmisión de señales a **larga distancia** (hasta 100 m).

4 Marco Metodológico

A continuación, se detalla la metodología de trabajo seguida para el desarrollo del proyecto, estructurada en fases semanales según el cronograma establecido. Cada fase se enfocó en el cumplimiento de objetivos específicos para asegurar una construcción modular y robusta del sistema de control.

Semana 1: Revisión bibliográfica y configuración del entorno (25/08 al 29/08):

Se realizó una investigación inicial sobre los principios de control para un péndulo invertido y se estudiaron las hojas de datos de los componentes clave: el microcontrolador ESP32, el driver de motor de paso DM542T y el encoder incremental. Se configuró el entorno de desarrollo Visual Studio Code con la extensión PlatformIO y el framework ESP-IDF, realizando pruebas de compilación y carga de código para verificar la comunicación con la placa.

Semana 2: Control básico de periféricos y calibración (01/09 al 05/09):

Se desarrollaron módulos de software independientes (librerías) para el control de bajo nivel. Se implementó una librería para el motor de paso capaz de generar trenes de pulsos a una frecuencia variable. Paralelamente, se creó una librería para el encoder incremental, utilizando el periférico de hardware PCNT del ESP32 en modo cuadratura para una lectura precisa del ángulo. Se finalizó con una rutina de calibración para encontrar los límites físicos del riel.

Semana 3: Implementación de la lógica de control (08/09 al 12/09):

Se diseñó la arquitectura del sistema de control principal. Se implementó inicialmente un controlador PID, estableciendo una tarea de alta prioridad en FreeRTOS para el bucle de control. Se realizaron pruebas en lazo abierto (comandos manuales) para verificar la respuesta del actuador y, posteriormente, se cerró el lazo utilizando la realimentación del encoder. Se inició el proceso de sintonización empírica de las ganancias del controlador.

Semana 4: Diseño de la interfaz de visualización (15/09 al 19/09):

Se integró el hardware de la pantalla LCD mediante su módulo adaptador I2C. Se desarrolló un módulo de software 'lcd_controller' para abstraer las operaciones de bajo nivel. Se diseñó una interfaz de usuario con múltiples vistas, gestionada por una tarea de baja prioridad, para mostrar en tiempo real el estado del sistema, el ángulo del péndulo, y las constantes del controlador, facilitando el diagnóstico y la sintonización.

- Semana 5: Integración del sistema (22/09 al 26/09):** Se unificaron todos los módulos desarrollados (controlador, motor, encoder, LCD, botones de interfaz) en una única aplicación concurrente. Se ajustaron las prioridades de las tareas para garantizar que el bucle de control en tiempo real no fuera afectado por las tareas de menor prioridad, como la actualización de la pantalla o la lectura de comandos UART.
- Semana 6: Optimización y pruebas de robustez (29/09 al 03/10):** Se realizó un proceso iterativo de sintonización fina de las ganancias del controlador para mejorar la estabilidad y la respuesta ante perturbaciones. Se implementaron optimizaciones en el código, como la migración del generador de pulsos al periférico MCPWM, y se realizaron pruebas de robustez para verificar el comportamiento del sistema en los límites de operación y la fiabilidad de los mecanismos de seguridad.
- Semana 7: Documentación del código (06/10 al 10/10):** Se llevó a cabo una revisión completa del código fuente para añadir comentarios explicativos en formato estándar. Se documentó la arquitectura general del software, el propósito de cada módulo (librería), la función de cada tarea de FreeRTOS y la interfaz pública de las funciones más importantes para facilitar su comprensión y futuro mantenimiento.
- Semana 8: Resultados y entrega final (13/10 al 17/10):** Se recopilaron los datos de rendimiento obtenidos durante las pruebas de robustez y se generaron gráficas para visualizar el comportamiento del sistema. Se preparó el material audiovisual para la presentación de resultados y se redactó el informe técnico final del proyecto, detallando el diseño, la implementación, las pruebas realizadas y las conclusiones.

5 Trabajo Realizado

5.1 Componentes

5.1.1 Motor de paso 23HS22-2804S STEPPERONLINE

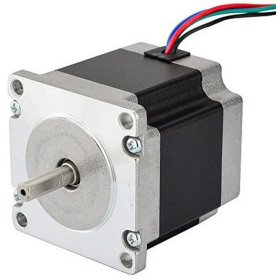


Figura 1 – Motor de paso

El 23HS22-2804S es un motor paso a paso (stepper motor) híbrido, bipolar, diseñado para operar en sistemas de dos fases.

Las características a tener en cuenta para su implementación en conjunto con el DM542T son:

- Corriente pico máxima de 2.8A (tabla 3).
- Torque máximo de 1.2Nm (tabla 3), según la carga a mover.
- Conexiones de las dos fases del motor descritas en la tabla 4.

5.1.2 Digital Stepper Drive DM542T.



Figura 2 – Controlador digital de motor de paso

El DM542T incluye las siguientes características clave según manual ([STEPPE-
RONLINE](#), 2020):

- Control de paso y dirección (PUL/DIR).
- Voltaje de entrada de 20-50VDC, con un rango recomendado de 24-48VDC.
- Frecuencia máxima de entrada de pulsos de 200 KHz.
- 16 resoluciones de micropasos de 200-25,600 configurables a través de interruptores DIP.
- 8 configuraciones de corriente de salida de 1.0-4.5A a través de interruptores DIP.
- Reducción de corriente en reposo (Idle current) seleccionable al 50% o 90% mediante el interruptor SW4.
- Auto-sintonización (*Auto-tuning*) para adaptarse a una amplia gama de motores paso a paso NEMA 11, 17, 23 y 24.
- Anti-Resonancia para un par óptimo, movimiento extra suave, y bajo calentamiento y ruido del motor.
- Arranque suave (*Soft-start*) sin "salto" al encenderse.
- Entradas ópticamente aisladas con opción de voltaje lógico de 5V o 24V.
- Salida de falla (*Fault output*).
- Protecciones integradas contra sobrevoltaje (activada cuando el voltaje de trabajo es mayor a 60VDC) y sobrecorriente.

5.1.3 ROTATORY ENCODER (INCREMENTAL) E6B2-CWZ6C OMRON Corporation.



(a) pines de alimentación.



(b) pines de fases.

Figura 3 – Encoder rotativo

Se trata de un Codificador Rotatorio Incremental de propósito general, con la siguiente descripción, según hoja de datos ([OMRON CORPORATION, INDUSTRIAL AUTOMATION COMPANY, 1992](#))

- Ofrece un **amplio rango de voltaje de operación de 5 a 24 VCC** (para el modelo de colector abierto).
- Posee una **resolución de 1024 pulsos/revolución** en una carcasa de 40 mm.
- La **Fase Z se puede ajustar con facilidad** utilizando la función de indicación de origen.
- Está disponible un modelo con **salida de controlador de línea (line driver)** que permite que el cable se extienda hasta 100 m.
- Se permite una **carga mecánica grande** en la dirección radial de 29.4 N (3 kgf) y en la dirección de empuje de 19.6 N (2 kgf).
- El dispositivo opera con **fases de salida A, B y Z** (reversible).
- La **frecuencia máxima de respuesta** es de 100 kHz para el modelo E6B2-CWZ6C.
- Incorpora un **circuito de protección** contra cortocircuito de carga y conexión inversa para asegurar una operación altamente fiable (aplicable al modelo de colector abierto con salida de voltaje).
- La **velocidad máxima de revolución permitida** es de 6,000 rpm.
- La **temperatura ambiente de operación** va de -10°C a 70°C (sin formación de hielo).
- Posee un **grado de protección IEC60529 IP50**, aunque no es resistente al agua ni al aceite [8].

Tabla 2 – Características del encoder.

Especificación	Valor
Resolución	1024P/R
Marrón	5 a 24 VDC
Azul	0V (Común)
Armadura	tierra (GND)
Negro	Salida A
Blanco	Salida B
Naranja	Salida Z

5.1.4 ESP32-WROOM-32



Figura 4 – ESP32-WROOM-32

El **ESP32-WROOM-32**, según ([ESPRESSIF SYSTEMS \(SHANGHAI\) CO., LTD., 2025](#)), es un módulo MCU (Microcontroller Unit) genérico y potente que integra conectividad Wi-Fi, Bluetooth y Bluetooth LE. Está diseñado para ser utilizado en una amplia gama de aplicaciones, que abarcan desde redes de sensores que requieren bajo consumo de energía hasta tareas más exigentes, como la codificación de voz, la transmisión de música (*music streaming*) y la decodificación de MP3.

En el núcleo del módulo se encuentra el chip **ESP32-D0WDQ6**, que utiliza un microprocesador Xtensa dual-core de 32 bits LX6, capaz de operar a una frecuencia de hasta 240 MHz. El módulo viene con una antena PCB integrada y 4 MB de flash SPI externo.

- **CPU y Memoria**

- Microprocesador Xtensa dual-core de 32 bits LX6, con hasta **240 MHz** de frecuencia.
- Incluye **448 KB de ROM**, **520 KB de SRAM** y **8 KB de SRAM** en el RTC (Reloj en Tiempo Real).
- Incorpora **4 MB de flash SPI**.

- **Conectividad Inalámbrica**

- **Wi-Fi**: Soporta estándares 802.11b/g/n, con una tasa de bits de hasta 150 Mbps (802.11n).
- **Bluetooth**: Compatible con especificaciones **Bluetooth V4.2 BR/EDR** y **Bluetooth LE** (Bajo Consumo).

- **Periféricos**

- Ofrece hasta **32 GPIOs** (incluyendo 5 GPIOs de *strapping*).
- Incluye interfaces **SD card**, **UART**, **SPI**, **SDIO**, **I2C**.

- Dispone de **controladores PWM** para LEDs y para Motores (Motor PWM).
- Incorpora **ADC** (Convertidor Analógico-Digital) y **DAC** (Convertidor Digital-Analógico).
- Cuenta con un **sensor táctil capacitivo**.
- Soporta la interfaz **TWAI®** (compatible con la Especificación CAN 2.0, según ISO 11898-1).

- **Condiciones de Operación**

- Voltaje de operación/alimentación: **3.0 3.6 V**.
- Temperatura ambiente de operación: de **-40 a 85 °C**.

Se incluye en los Anexos el pinout correspondiente al utilizado en el trabajo (fig. 19) y el link correspondiente a la hoja de datos del mismo en la bibliografía ([ESPRESSIF SYSTEMS \(SHANGHAI\) CO., LTD., 2025](#)).

5.1.5 Pantalla LCD WH1602B 16 caracteres x 2 líneas.



Figura 5 – Pantalla LCD.

El módulo de visualización WH1602B ([WINSTAR DISPLAY CO.,](#)) es una pantalla LCD que presenta un formato de 16 caracteres por 2 líneas. Sus dimensiones son de 80.0 x 36.0 x 13.5 mm (máximo), con un área de visualización (View area) de 66.0 x 16.0 mm. Utiliza un backlight de tipo LED y está controlado por el circuito integrado IC ST7066U, comunicándose mediante una interfaz de la serie 68. En cuanto a sus requisitos eléctricos, opera con un voltaje de suministro lógico VDD-VSS de 4.5 a 5.5 V (típico 5.0 V), y está diseñada para funcionar en un amplio rango de temperatura operativa de -20 °C a +70 °C.

5.1.6 Adaptador I2C para pantalla LCD.

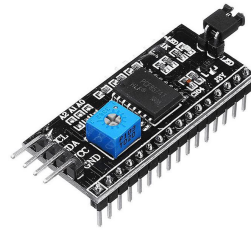


Figura 6 – Adaptador I2C.

El modulo consiste en un diseño que utiliza el chip MCP23017 y es compatible con pantallas de cristal líquido de 1602 y 12864, a través de este módulo se cambia el bus habitual de comunicación a el uso I2C, para ello solo se necesitan dos I/Os del MCU para poder realizar el control de la pantalla de cristal líquido. El potenciómetro del módulo permite realizar un ajuste de contraste. También posee un jumper que controla el encendido de la retroiluminación o ajustar el brillo a través de PWM.

5.1.7 Convertidor DC DC Arduino basado en LM2596.



Figura 7 – convertidor LM2596.

El convertidor de voltaje DC-DC Step-Down 3A LM2596 tiene como función entregar un voltaje de salida constante inferior al voltaje de entrada frente a variaciones del voltaje de entrada o de carga. Soporta corrientes de salida de hasta 3A, voltaje de entrada entre 4.5V a 40V y voltaje de salida entre 1.23V a 37V. El voltaje de salida se selecciona mediante un potenciómetro multivuelta.

5.2 Implementación de Encoder

5.2.1 Caracterización y conexión del encoder

Se han preparado dos figuras basadas en la hoja de datos ([OMRON CORPORATION, INDUSTRIAL AUTOMATION COMPANY, 1992](#)): una que muestra el diagrama

interno de salida/conexión a colector abierto (figura 8) y otra que muestra la señal esperada (figura 9) entre las fases A, B y Z.

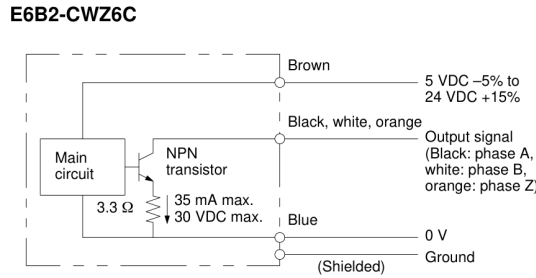


Figura 8 – Circuito de salida del encoder (basado en hoja de datos E6B2-CWZ6C). Los pines: marrón = Vcc (5–24 V), azul = GND, negro = fase A, blanco = fase B, naranja = fase Z.

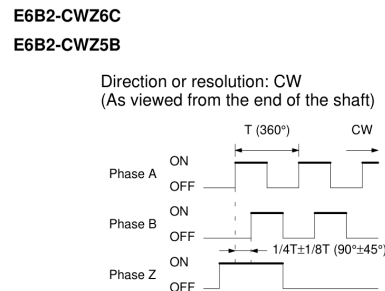


Figura 9 – Señales esperadas en las fases A, B y Z (timing y desfase) por salida de colector abierto.

Las fases A y B entregan pulsos en cuadratura con un desfase de $90^\circ (\pm 45^\circ)$. La fase Z produce un pulso por revolución que permite el establecimiento de la referencia absoluta de giro. El modelo E6B2-CWZ6C admite salidas de colector abierto con máximo de 35 mA y tensión hasta 30 VDC (ver figura 8).

5.2.2 Montaje y señal observada

Para validar el funcionamiento del encoder se realizó un montaje de prueba sencillo: alimentación estable (5 VDC), que puede tomarse directo del pin 5V del esp32 o de una fuente externa; las salidas A/B/Z conectadas a entradas de captura mediante resistencias pull-up (cuando es necesario) y la rotación manual del eje.

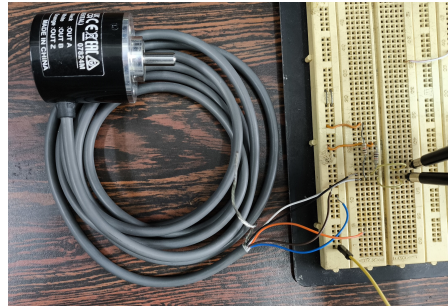


Figura 10 – Montaje de prueba del encoder: conexiones a la fuente de 5 V y sondas de osciloscopio en A y B.

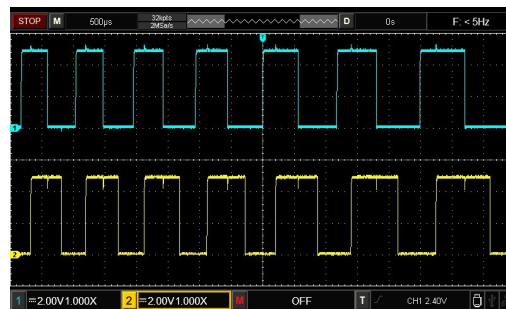


Figura 11 – Señal observada en prueba: canales A y B en cuadratura.

La señal de salida presenta niveles TTL compatibles con entradas de microcontrolador cuando se alimenta a 5V. A velocidades bajas la forma de onda es estable y limpia; a mayor velocidad se observan pequeñas transitorias despreciables.

5.2.3 Integración con ESP32: lectura y salida de datos

En la fase de integración se conectó el encoder a entradas GPIO del ESP32, basándose en el diagrama de conexiones de la figura 17. Se implementó un programa de lectura en el ESP32 que cuenta pulsos, determina dirección y reporta la posición por puerto serie; indicado en la sección 6.1.

La figura 12 muestra el montaje físico del encoder conectado al ESP32. La figura 13 muestra una captura de la salida serial (ejemplo) que también es de terminal, generada por el firmware de lectura.

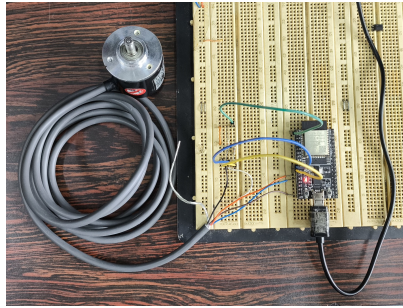


Figura 12 – Montaje físico del encoder conectado al ESP32 durante las pruebas de integración.

```
I (286) main.cpp: calling app_main()<CR><LF>
I (286) ENCODER_LIB: Librería Encoder inicializada en pines A127, B114, Z112<CR><LF>
I (286) MAIN_APP: Angulo del Encoder: 0.00 grados<CR><LF>
I (786) MAIN_APP: Angulo del Encoder: 0.00 grados<CR><LF>
I (1286) MAIN_APP: Angulo del Encoder: 27.42 grados<CR><LF>
I (1786) MAIN_APP: Angulo del Encoder: 64.16 grados<CR><LF>
I (2286) MAIN_APP: Angulo del Encoder: 140.15 grados<CR><LF>
I (2786) MAIN_APP: Angulo del Encoder: 33.22 grados<CR><LF>
W (2786) MAIN_APP: ¡Pulso de INDICE (Z) detectado! El contador se ha reseteado.<CR><LF>
I (3286) MAIN_APP: Angulo del Encoder: 79.06 grados<CR><LF>
I (3786) MAIN_APP: Angulo del Encoder: 24.26 grados<CR><LF>
I (4286) MAIN_APP: Angulo del Encoder: -59.59 grados<CR><LF>
W (4286) MAIN_APP: ¡Pulso de INDICE (Z) detectado! El contador se ha reseteado.<CR><LF>
I (4786) MAIN_APP: Angulo del Encoder: -153.11 grados<CR><LF>
I (5286) MAIN_APP: Angulo del Encoder: -237.83 grados<CR><LF>
I (5786) MAIN_APP: Angulo del Encoder: -240.01 grados<CR><LF>
I (6286) MAIN_APP: Angulo del Encoder: -109.16 grados<CR><LF>
I (6786) MAIN_APP: Angulo del Encoder: -12.38 grados<CR><LF>
I (7286) MAIN_APP: Angulo del Encoder: 53.96 grados<CR><LF>
W (7286) MAIN_APP: ¡Pulso de INDICE (Z) detectado! El contador se ha reseteado.<CR><LF>
I (7786) MAIN_APP: Angulo del Encoder: 88.33 grados<CR><LF>
I (8286) MAIN_APP: Angulo del Encoder: 58.10 grados<CR><LF>
I (8786) MAIN_APP: Angulo del Encoder: 37.62 grados<CR><LF>
I (9286) MAIN_APP: Angulo del Encoder: 10.20 grados<CR><LF>
I (9786) MAIN_APP: Angulo del Encoder: -12.74 grados<CR><LF>
W (9786) MAIN_APP: ¡Pulso de INDICE (Z) detectado! El contador se ha reseteado.<CR><LF>
I (10286) MAIN_APP: Angulo del Encoder: -47.02 grados<CR><LF>
I (10786) MAIN_APP: Angulo del Encoder: -144.93 grados<CR><LF>
I (11286) MAIN_APP: Angulo del Encoder: -267.01 grados<CR><LF>
```

Figura 13 – Captura de la salida serial o terminal del ESP32 mostrando conteo de pulsos, dirección y detección de Z.

Observación: Para la prueba realizada se utilizó una resistencia de pull-up para la fase A y B, sin embargo Z no la incorporó para probar la activación de pull-up interno del esp32 por software, es decir, estas resistencias no son necesarias, siempre y cuando se active el pull-up interno del esp32; aunque es necesario incorporarlas para evitar activaciones por ruidos externos, según la aplicación.

5.3 Implementación del driver y motor

En esta sección se documenta la integración del controlador de motor paso a paso DM542T con el motor 23HS22-2804S y su control desde el ESP32. El montaje se realizó siguiendo el diagrama de conexiones (ver figura 18 en los resultados) donde se muestran las señales PUL (pulsos), DIR (dirección) y ENA (habilitación).

5.3.1 Conexiones y consideraciones de señal

El DM542T acepta entradas ópticamente aisladas para PUL, DIR y ENA; puede funcionar con niveles lógicos compatibles con 5 V cuando el aislamiento está configurado apropiadamente. En la integración con el ESP32 se utilizó la siguiente convención (ejemplo):

- PUL: recibe el tren de pulsos que hace avanzar el motor.

- DIR: selecciona el sentido de giro (hora/anti-hora).
- ENA: señal opcional para habilitar o deshabilitar el driver.

En el diagrama de conexiones usado (14) se observa la ruta desde el ESP32 a las entradas del DM542T. Es importante respetar la polaridad y los jumpers de configuración del driver (niveles y pull-ups) y emplear una masa común cuando no se usa el aislamiento óptico.

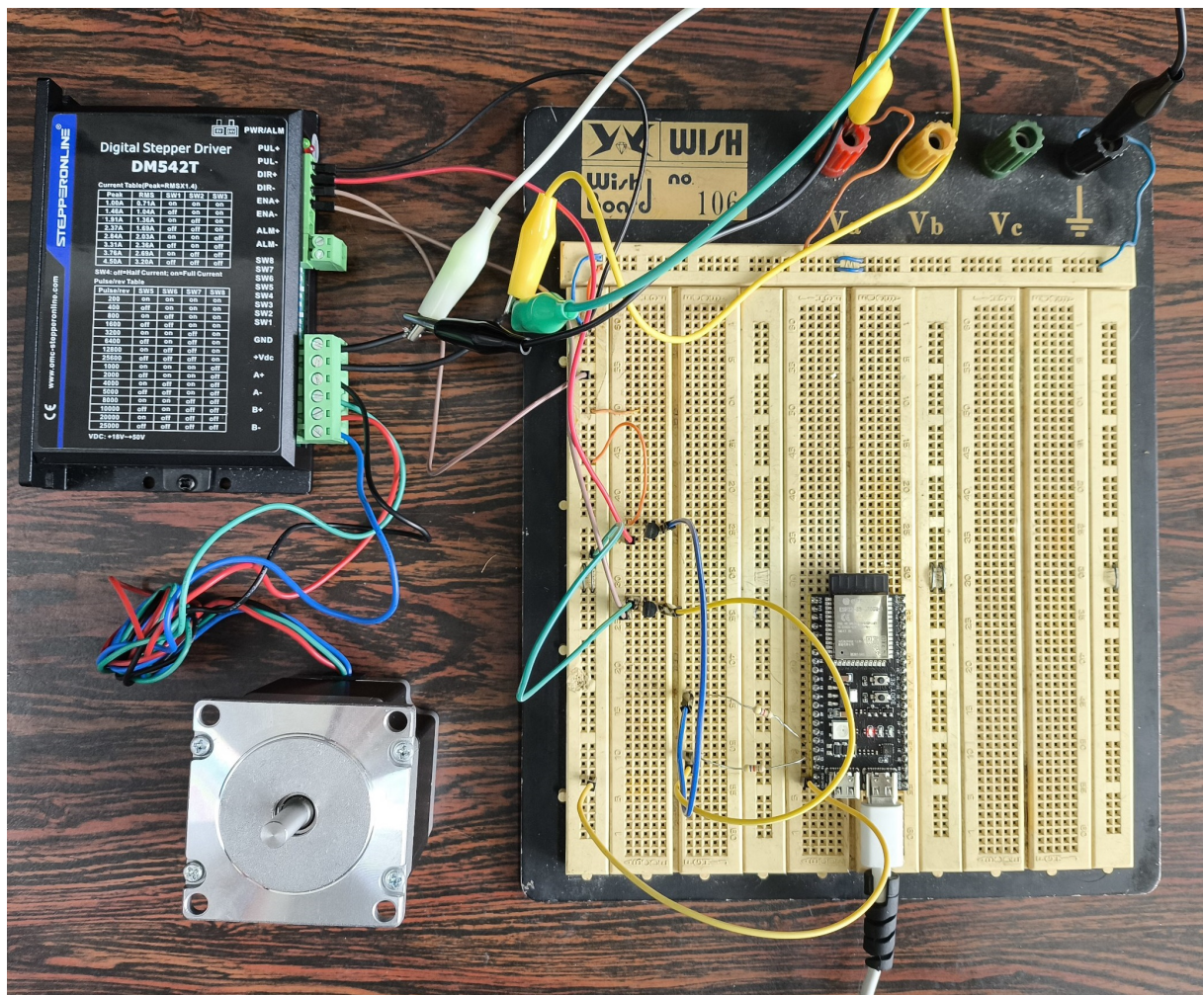


Figura 14 – Diagrama de conexiones driver y motor, en conjunto con el ESP32.

5.3.2 Cálculo de pulsos por revolución y configuración de microstepping

Para controlar exactamente una vuelta del eje se debe enviar al pin PUL un número de pulsos igual al número de pasos por revolución del motor multiplicado por el factor de microstepping configurado en el DM542T.

Fórmula general:

5.3.3 Frecuencia de pulsos: mínimos y máximos

El DM542T soporta una frecuencia de pulsos de hasta 200 kHz (según la hoja de datos) y para una operación confiable en este montaje se utilizó como frecuencia mínima práctica 10 kHz. Con estos límites, el tiempo requerido para una revolución completa queda:

- A 10 kHz: tiempo = $\text{Pulsesperrev} / 10000 = 3200 / 10,000 = 0.32 \text{ s} \rightarrow \text{RPM} \approx 187.5 \text{ rpm}$. - A 200 kHz: tiempo = $3200 / 200000 = 0.016 \text{ s} \rightarrow \text{RPM} \approx 3750 \text{ rpm}$.

Estos valores son orientativos: la velocidad segura depende de la carga, el par requerido y las limitaciones térmicas del motor y driver. Ajuste la corriente del DM542T con los dip switches al valor nominal del motor y emplee microstepping según la necesidad de resolución/torque.

5.3.4 Generación de pulsos desde el ESP32

Para frecuencias altas ($\gg 10 \text{ kHz}$) es recomendable usar periféricos de hardware del ESP32 para generar el tren de pulsos:

- RMT (Remote Control) o PWM/hardware timer para generar pulsos en hardware, evitando la pérdida de pulsos por bloqueo en el scheduler. - Evitar usar delays software largos o loops con digitalWrite a altas frecuencias (riesgo de jitter y pérdida).

En el control de una vuelta el algoritmo básico es:

1. Configurar GPIO DIR al nivel deseado.
2. Habilitar ENA (si se usa).
3. Generar $N = \text{Pulsesperrev}$ pulsos en PUL con la frecuencia deseada.
4. Deshabilitar ENA (opcional).

5.3.5 Prueba de una vuelta — procedimiento y observaciones

Procedimiento realizado:

1. Configurar el DM542T a 1/16 microstepping y corriente adecuada para el motor.
2. Conectar PUL a un pin del ESP32 capaz de salida PWM/RMT y DIR a un GPIO digital.
3. Generar 3200 pulsos a 10 kHz y verificar que el eje completa una revolución.
4. Repetir a 50 kHz y observar comportamiento; finalmente probar en el rango hasta 200 kHz para verificar respuesta.

Observaciones recomendadas:

- Verificar que la corriente de ajuste del driver sea la adecuada (evitar sobrecalentamiento). - Observar vibraciones a microsteppings bajos y calor del motor a altas frecuencias. - Si se experimenta pérdida de pasos, reducir la velocidad, aumentar corriente o mejorar la aceleración (rampado) de la secuencia de pulsos.

5.4 Diagrama de Conexiones.

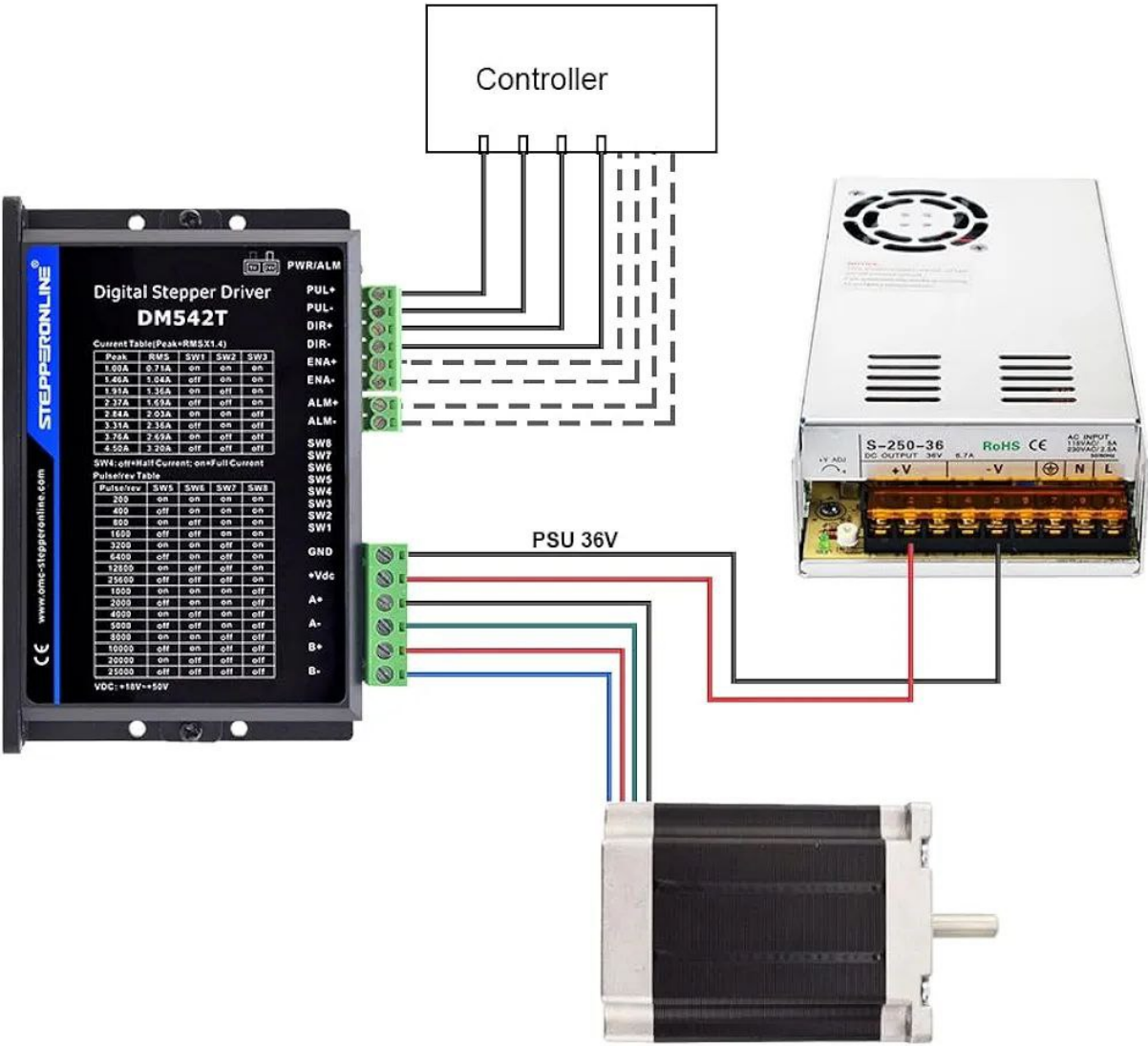


Figura 15 – Diagrama de conexiones para utilizar motor.

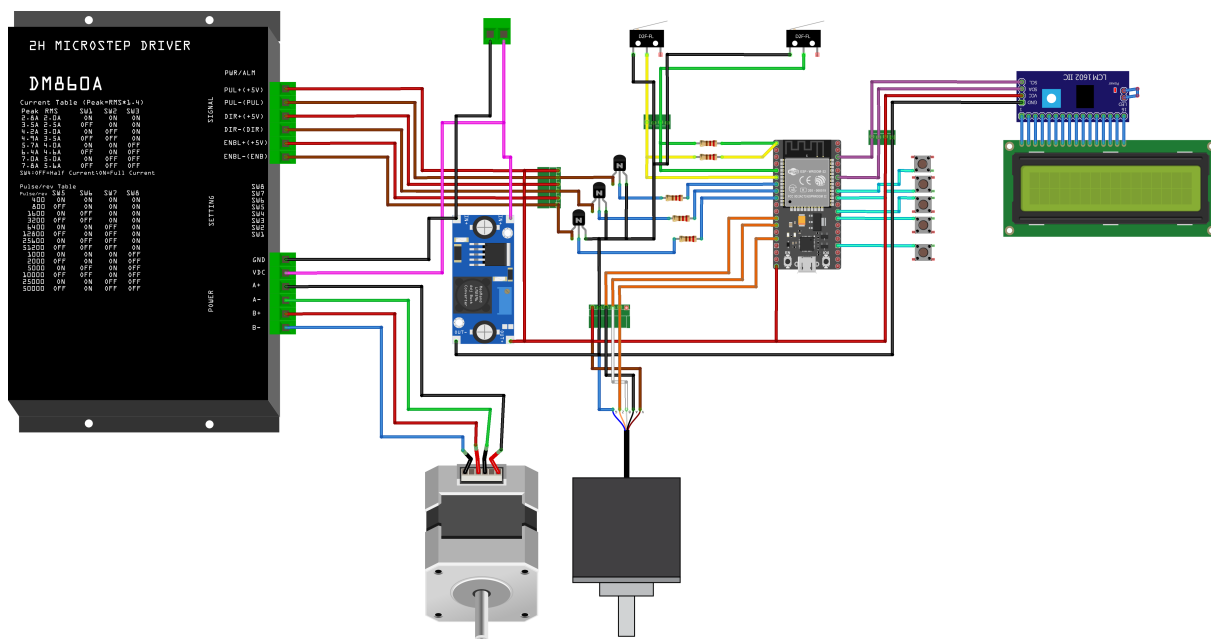


Figura 16 – Diagrama de conexiones de sistema.

6 Resultados

6.1 Encoder

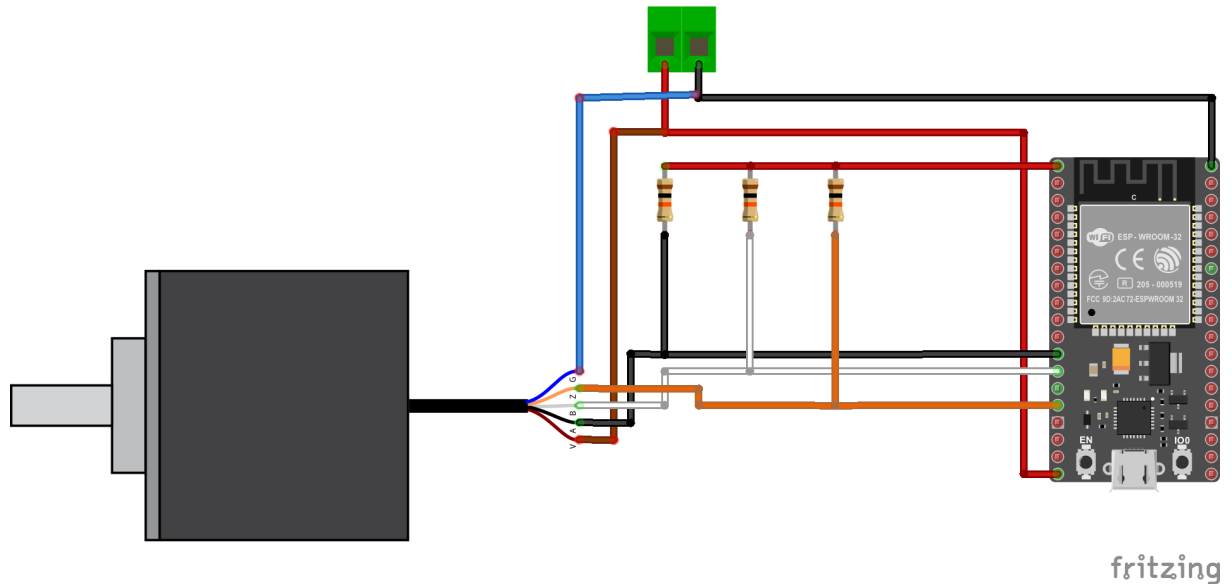


Figura 17 – Diagrama de conexiones de integración encoder y esp32.

```
// src/main.c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "encoder.h" // ¡Solo incluimos nuestra nueva librería!

// --- CONFIGURACIÓN DEL USUARIO ---
#define ENCODER_PIN_A 27
#define ENCODER_PIN_B 14
#define ENCODER_PIN_Z 12
#define ENCODER_PULSES_PER_REV 1024

static const char *TAG = "MAIN_APP";

void app_main(void) {
    // 1. Rellenar la estructura de configuración
    encoder_config_t encoder_cfg = {
        .pin_a = ENCODER_PIN_A,
        .pin_b = ENCODER_PIN_B,
```

```
.pin_z = ENCODER_PIN_Z,
.pulses_per_rev = ENCODER_PULSES_PER_REV
};

// 2. Inicializar la librería
esp_err_t ret = encoder_init(&encoder_cfg);
if (ret != ESP_OK) {
    ESP_LOGE(TAG, "Fallo al inicializar el encoder, deteniendo.");
    return;
}

// 3. Bucle principal para mostrar los datos
while (1) {
    // Obtener el ángulo actual usando la función de la librería
    float angle = encoder_get_angle_degrees();

    // Imprimir el ángulo
    ESP_LOGI(TAG, "Angulo del Encoder: %.2f grados", angle);

    // Comprobar si se ha detectado el pulso Z
    if (encoder_was_z_pulse_detected()) {
        ESP_LOGW(TAG, "¡Pulso de INDICE (Z) detectado! El contador se ha reseteado");
    }

    vTaskDelay(pdMS_TO_TICKS(500)); // Mostrar datos 1 vez por segundo
}
}
```

6.2 Motor

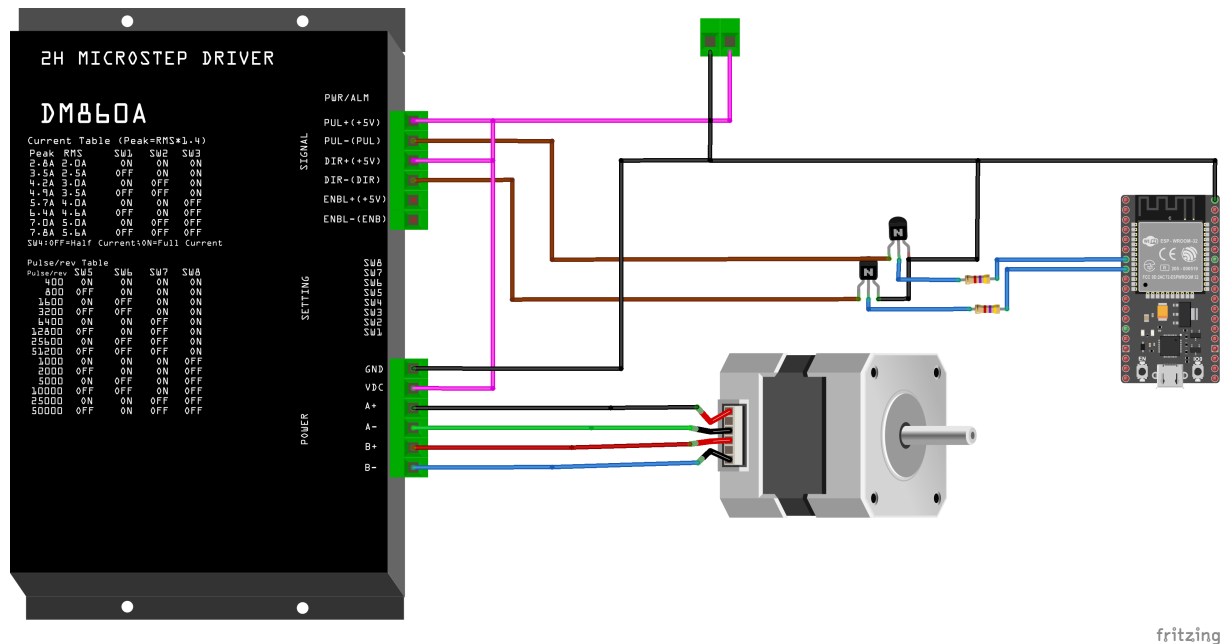


Figura 18 – Diagrama de conexiones driver y motor, en conjunto con el esp32.

```
// src/main.c

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "stepperMotor.h" // ¡Solo incluimos nuestra nueva librería!
#include "esp_log.h"
#include "uart_echo.h"

#define STEP_PIN 12
#define DIR_PIN 13

void mov_continuo(void);
void mov_manual(void);

void app_main(void) {

    // 1. Configurar la librería con los pines que queremos usar
    stepper_config_t motor_cfg = {
        .step_pin = STEP_PIN,
        .dir_pin = DIR_PIN
    };

    // 2. Inicializar la librería
```

```
esp_err_t ret = stepper_init(&motor_cfg);
if (ret != ESP_OK) {
    ESP_LOGE("MAIN", "Fallo al inicializar el motor, deteniendo.");
    return;
}

// 3. ¡Usar la librería!
ESP_LOGI("MAIN", "Iniciando movimiento de prueba...");

//-----Escoger una de las opciones, comentar la otra-----

// OPCION A: movimiento continuo
mov_continuo();

// OPCION B: movimiento manual (por comandos UART)
//mov_manual();

}

void mov_continuo(void) {
    while (1) {

        // Mover 10000 pasos (una vuelta) a 10kHz en dirección 1
        ESP_LOGI("MAIN", "Moviendo a la derecha...");
        stepper_move(10000, 10000, 1);
        vTaskDelay(pdMS_TO_TICKS(1000)); // Pausa de 1 segundo

        // Mover 10000 pasos (dos vueltas) a 10kHz en dirección 0
        ESP_LOGI("MAIN", "Moviendo a la izquierda más rápido...");
        stepper_move(20000, 10000, 0);
        vTaskDelay(pdMS_TO_TICKS(1000));

    }
}

void mov_manual(void) {
    uart_init(); // Inicializa el uart

    uint8_t *data = (uint8_t *) malloc(BUF_SIZE);
```

```

if (data == NULL) {
    ESP_LOGE("MAIN", "No se pudo asignar memoria para el buffer UART");
    vTaskDelete(NULL);
}
ESP_LOGI("MAIN", "Formato: PULSOS <#pulsos> <freq> <dir>. Ej: PULSOS 200 1000 1

while (1) {
    int len = uart_read_bytes(UART_PORT, data, (BUF_SIZE - 1), 20 / portTICK_PERIOD_MS);
    if (len > 0) {
        data[len] = '\0';
        ESP_LOGI("MAIN", "Ejecutando: %s", (char *) data);
        //uart_write_bytes(UART_PORT, (const char *) data, len); // Eco de lo recibido

        // --- LÓGICA DE PARSEO ---
        char *cmd_token = strtok((char *)data, " ");
        if (cmd_token != NULL && strcmp(cmd_token, "PULSOS") == 0) {

            char *pulses_token = strtok(NULL, " ");
            char *freq_token = strtok(NULL, " ");
            char *dir_token = strtok(NULL, " \r\n");

            if (pulses_token != NULL && freq_token != NULL && dir_token != NULL) {
                int num_pulses = atoi(pulses_token);
                int frequency = atoi(freq_token);
                int direction = atoi(dir_token);

                // Validar los datos recibidos
                if (num_pulses > 0 && frequency > 0 && (direction == 0 || direction == 1)) {
                    stepper_move(num_pulses, frequency, direction);
                } else {
                    uart_write_bytes(UART_PORT, "Error: Argumentos inválidos.\r\n", strlen("Error: Argumentos inválidos.\r\n"));
                }
            } else {
                uart_write_bytes(UART_PORT, "Error: Faltan argumentos. Formato: PULSOS <#pulsos> <freq> <dir>.\r\n", strlen("Error: Faltan argumentos. Formato: PULSOS <#pulsos> <freq> <dir>.\r\n"));
            }
        } else {
            uart_write_bytes(UART_PORT, "Comando desconocido.\r\n", strlen("Comando desconocido.\r\n"));
        }
    }
}

```


}
}

7 Conclusión

Conclusion.

Referencias

ESPRESSIF SYSTEMS (SHANGHAI) CO., LTD. **ESP32-WROOM-32 Datasheet Version 3.6**. Shanghai, China, 2025. Documento de especificaciones técnicas (Datasheet). Disponible em: <https://espressif.com/documentation/esp32-wroom-32_datasheet_en.pdf>.

LEILI MOTOR. **Motores paso a paso unipolares vs bipolares: ¿cuál es más eficiente?** No. 19 Qianjiatang Road, Wujin District, Changzhou City, Jiangsu Province, 2022. Artículo de comparación técnica. Copyright 2022: Todos los derechos reservados.

OMRON CORPORATION, INDUSTRIAL AUTOMATION COMPANY. **Rotary Encoder E6B2: New General-purpose Incremental Rotary Encoder**. Osaka, Japan, 1992. Documento de Especificaciones Técnicas. Cat. No. Q085-E1-1C. Código de impresión (1092). Disponible em: <<https://docs.rs-online.com/35ca/0900766b80383981.pdf>>.

STEPPERONLINE. **User Manual DM542T(V4.0) 2-Phase Digital Stepper Drive**. Revision 4.0. [S.l.], 2020. Disponible em: <https://www.omc-stepperonline.com/download/DM542T_V4.0.pdf>.

WEST INSTRUMENTS DE MÉXICO, S.A. **MANUAL DE APLICACIÓN DE ENCODERS: Una guía de referencia y tutoría sobre encoders para control de movimiento: Tipos, tecnologías, aplicaciones, e instalaciones**. [S.l.]. Disponible em: <www.westmexico.com.mx>.

WINSTAR DISPLAY CO. **SPECIFICATION MODULE NO.: WH1602B General Specification**. Taiwan (Ubicación de la compañía). Módulo LCD de 16 caracteres x 2 líneas con IC ST7066U e interfaz de la serie 68. Disponible em: <<https://www.winstar.com.tw/products/lcd-display/character-lcd-display-module/lcd-display-16x2.html>>.

Apêndices

APÊNDICE A – Códigos

A.1 Códigos librería encoder

```
// lib/encoder/encoder.h
#ifndef ENCODER_H
#define ENCODER_H

#include "driver/gpio.h"
#include <stdbool.h>

/**
 * @brief Estructura de configuración para inicializar el encoder.
 */
typedef struct {
    gpio_num_t pin_a;           // Pin GPIO para la Fase A
    gpio_num_t pin_b;           // Pin GPIO para la Fase B
    gpio_num_t pin_z;           // Pin GPIO para la Fase Z (índice), usar GPIO_NUM_13
    uint16_t pulses_per_rev;     // Pulsos por revolución del encoder (ej. 1024)
} encoder_config_t;

/**
 * @brief Inicializa el controlador del encoder.
 *
 * Configura el periférico PCNT y las interrupciones de GPIO.
 * Debe ser llamada una sola vez.
 *
 * @param config Puntero a la estructura de configuración.
 * @return esp_err_t ESP_OK si la inicialización fue exitosa.
 */
esp_err_t encoder_init(const encoder_config_t *config);

/**
 * @brief Obtiene el valor crudo actual del contador del encoder.
 *
 * El rango dependerá del contador de 16 bits del PCNT.
 */
```

```
* @return El valor actual del contador.
*/
int16_t encoder_get_counts(void);

/**
 * @brief Obtiene el ángulo actual del encoder en grados.
 *
 * Calcula el ángulo basado en la resolución y el conteo actual.
 *
 * @return El ángulo en grados (-180.0 a 180.0).
 */
float encoder_get_angle_degrees(void);

/**
 * @brief Comprueba si se ha detectado un pulso de índice (Z) desde la última vez q
 *
 * Esta función es no bloqueante y resetea el flag al ser leída.
 *
 * @return true si se detectó un pulso Z, false si no.
 */
bool encoder_was_z_pulse_detected(void);

#endif // ENCODER_H

// lib/encoder/encoder.c
#include "encoder.h"
#include "freertos/FreeRTOS.h"
#include "driver/pcnt.h"
#include "esp_log.h"
#include "esp_err.h"

#define PCNT_UNIT PCNT_UNIT_0
static const char *TAG = "ENCODER_LIB";

// Variables estáticas de la librería
static encoder_config_t g_config;
static float g_degrees_per_count;
static volatile bool g_z_pulse_flag = false;
```

```
// ISR para la señal Z
static void IRAM_ATTR encoder_z_isr_handler(void* arg) {
    pcnt_counter_clear(PCNT_UNIT);
    g_z_pulse_flag = true;
}

esp_err_t encoder_init(const encoder_config_t *config) {
    if (config == NULL) return ESP_ERR_INVALID_ARG;
    g_config = *config;

    // Calcular el factor de conversión una sola vez
    g_degrees_per_count = 360.0f / (float)(g_config.pulses_per_rev * 4);

    // Configuración del Canal 0 (A y B)
    pcnt_config_t pcnt_config_ch0 = {
        .pulse_gpio_num = g_config.pin_a,
        .ctrl_gpio_num = g_config.pin_b,
        .channel = PCNT_CHANNEL_0,
        .unit = PCNT_UNIT,
        .pos_mode = PCNT_COUNT_DEC,
        .neg_mode = PCNT_COUNT_INC,
        .lctrl_mode = PCNT_MODE_REVERSE,
        .hctrl_mode = PCNT_MODE_KEEP,
    };
    ESP_ERROR_CHECK(pcnt_unit_config(&pcnt_config_ch0));

    // Configuración del Canal 1 (B y A)
    pcnt_config_t pcnt_config_ch1 = {
        .pulse_gpio_num = g_config.pin_b,
        .ctrl_gpio_num = g_config.pin_a,
        .channel = PCNT_CHANNEL_1,
        .unit = PCNT_UNIT,
        .pos_mode = PCNT_COUNT_INC,
        .neg_mode = PCNT_COUNT_DEC,
        .lctrl_mode = PCNT_MODE_REVERSE,
        .hctrl_mode = PCNT_MODE_KEEP,
    };
    ESP_ERROR_CHECK(pcnt_unit_config(&pcnt_config_ch1));
```

```
gpio_set_pull_mode(g_config.pin_a, GPIO_PULLUP_ONLY);
gpio_set_pull_mode(g_config.pin_b, GPIO_PULLUP_ONLY);

pcnt_set_filter_value(PCNT_UNIT, 100);
pcnt_filter_enable(PCNT_UNIT);

pcnt_counter_pause(PCNT_UNIT);
pcnt_counter_clear(PCNT_UNIT);
pcnt_counter_resume(PCNT_UNIT);

// Configurar la interrupción Z solo si se ha proporcionado un pin válido
if (g_config.pin_z != GPIO_NUM_NC) {
    gpio_config_t z_pin_config = {
        .pin_bit_mask = (1ULL << g_config.pin_z),
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE,
        .intr_type = GPIO_INTR_POSEDGE
    };
    gpio_config(&z_pin_config);
    gpio_install_isr_service(0);
    gpio_isr_handler_add(g_config.pin_z, encoder_z_isr_handler, NULL);
}

ESP_LOGI(TAG, "Librería Encoder inicializada en pines A:%d, B:%d, Z:%d",
          g_config.pin_a, g_config.pin_b, g_config.pin_z);
return ESP_OK;
}

int16_t encoder_get_counts(void) {
    int16_t count = 0;
    pcnt_get_counter_value(PCNT_UNIT, &count);
    return count;
}

float encoder_get_angle_degrees(void) {
    int16_t count = 0;
    pcnt_get_counter_value(PCNT_UNIT, &count);
    return (float)count * g_degrees_per_count;
}
```



```
bool encoder_was_z_pulse_detected(void) {
    if (g_z_pulse_flag) {
        g_z_pulse_flag = false; // Resetear el flag al leerlo
        return true;
    }
    return false;
}
```

A.2 Códigos librería stepperMotor

```
// lib/stepperMotor/stepperMotor.h

#ifndef STEPPER_MOTOR_H
#define STEPPER_MOTOR_H

#include "driver/gpio.h"

/**
 * @brief Estructura de configuración para inicializar el motor a pasos.
 * El usuario debe rellenar esta estructura con los pines que va a utilizar.
 */
typedef struct {
    gpio_num_t step_pin;    // Pin GPIO para los pulsos (STEP/PUL)
    gpio_num_t dir_pin;     // Pin GPIO para la dirección (DIR)
} stepper_config_t;

/**
 * @brief Inicializa el controlador del motor a pasos.
 *
 * Configura los periféricos de hardware (LEDC/MCPWM) y los pines GPIO necesarios.
 * Debe ser llamada una sola vez antes de cualquier otra función de la librería.
 *
 * @param config Puntero a la estructura de configuración con los pines a utilizar.
 * @return esp_err_t ESP_OK si la inicialización fue exitosa, o un código de error.
 */
esp_err_t stepper_init(const stepper_config_t *config);

/**
```

```
* @brief Mueve el motor un número específico de pasos a una velocidad dada.
*
* Esta es una función bloqueante. La ejecución no continuará hasta que el
* movimiento haya terminado.
*
* @param steps El número de micropasos a mover.
* @param speed_hz La velocidad del movimiento en Hertz (pasos por segundo).
* @param direction La dirección del movimiento (0 o 1).
* @return esp_err_t ESP_OK si el comando es válido.
*/
esp_err_t stepper_move(int steps, int speed_hz, int direction);

#endif // STEPPER_MOTOR_H

// lib/stepperMotor/stepperMotor.c

#include "stepperMotor.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/ledc.h"
#include "esp_log.h"
#include "esp_err.h"

// --- Definiciones privadas de la librería ---
#define LEDC_TIMER LEDC_TIMER_0
#define LEDC_MODE LEDC_LOW_SPEED_MODE
#define LEDC_CHANNEL LEDC_CHANNEL_0
#define LEDC_DUTY_RES LEDC_TIMER_1_BIT // Óptimo para alta frecuencia

static const char *TAG = "STEPPER_MOTOR";
static stepper_config_t motor_config; // Variable global para guardar los pines

esp_err_t stepper_init(const stepper_config_t *config) {
    if (config == NULL) {
        ESP_LOGE(TAG, "La configuración no puede ser NULL");
        return ESP_ERR_INVALID_ARG;
    }
    motor_config = *config; // Copiar la configuración del usuario
```

```
ESP_LOGI(TAG, "Inicializando motor en STEP pin %d, DIR pin %d", motor_config.step_pin, motor_config.dir_pin);

// Configuración del temporizador LEDC
ledc_timer_config_t ledc_timer = {
    .speed_mode      = LEDC_MODE,
    .timer_num       = LEDC_TIMER,
    .duty_resolution = LEDC_DUTY_RES,
    .freq_hz         = 10000, // Frecuencia inicial minima
    .clk_cfg         = LEDC_AUTO_CLK
};

esp_err_t err = ledc_timer_config(&ledc_timer);
if (err != ESP_OK) {
    ESP_LOGE(TAG, "Fallo al configurar el temporizador LEDC: %s", esp_err_to_name(err));
    return err;
}

// Configuración del canal PWM (STEP pin)
ledc_channel_config_t ledc_channel = {
    .speed_mode      = LEDC_MODE,
    .channel         = LEDC_CHANNEL,
    .timer_sel       = LEDC_TIMER,
    .intr_type       = LEDC_INTR_DISABLE,
    .gpio_num        = motor_config.step_pin,
    .duty            = 0,
    .hpoint          = 0
};

err = ledc_channel_config(&ledc_channel);
if (err != ESP_OK) {
    ESP_LOGE(TAG, "Fallo al configurar el canal LEDC: %s", esp_err_to_name(err));
    return err;
}

// Configuración del pin de dirección (DIR pin)
gpio_config_t dir_gpio_config = {
    .pin_bit_mask = (1ULL << motor_config.dir_pin),
    .mode         = GPIO_MODE_OUTPUT
};

err = gpio_config(&dir_gpio_config);
if (err != ESP_OK) {
```

```
        ESP_LOGE(TAG, "Fallo al configurar el pin de dirección: %s", esp_err_to_name(err));
        return err;
    }

    ESP_LOGI(TAG, "Librería Stepper Motor inicializada correctamente.");
    return ESP_OK;
}

esp_err_t stepper_move(int steps, int speed_hz, int direction) {
    if (steps <= 0 || speed_hz <= 0) {
        return ESP_ERR_INVALID_ARG;
    }

    // 1. Establecer la dirección
    gpio_set_level(motor_config.dir_pin, direction);
    esp_rom_delay_us(10); // Pequeña espera para que el driver asiente la dirección

    // 2. Ajustar la frecuencia del PWM
    esp_err_t err = ledc_set_freq(LED_MODE, LEDC_TIMER, speed_hz);
    if (err != ESP_OK) {
        ESP_LOGE(TAG, "Fallo al establecer la frecuencia a %d Hz: %s", speed_hz, esp_err_to_name(err));
        return err;
    }

    // 3. Calcular la duración
    uint32_t duration_ms = (uint32_t)(steps * 1000) / speed_hz;
    if (duration_ms == 0) duration_ms = 1; // Mover al menos por un tick

    // 4. Iniciar la generación de pulsos (50% duty cycle)
    const uint32_t duty = (1 << LEDC_DUTY_RES) / 2;
    ledc_set_duty(LED_MODE, LEDC_CHANNEL, duty);
    ledc_update_duty(LED_MODE, LEDC_CHANNEL);

    // 5. Esperar el tiempo calculado
    vTaskDelay(pdMS_TO_TICKS(duration_ms));

    // 6. Detener la generación de pulsos
    ledc_set_duty(LED_MODE, LEDC_CHANNEL, 0);
    ledc_update_duty(LED_MODE, LEDC_CHANNEL);
}
```

```
// Opcional: poner la dirección en un estado por defecto
gpio_set_level(motor_config.dir_pin, 0);

return ESP_OK;
}
```

Anexos

ANEXO A – Información sobre Motor de paso 23HS22-2804S

Tabla 3 – Tabla de Especificaciones (Specification)

SPECIFICATION / CONNECTION	BIPOLAR
AMPS/PHASE	2.80
RESISTANCE/PHASE (Ohms)@25°C	$0.90 \pm 10\%$
INDUCTANCE/PHASE (mH)@1kHz	$2.86 \pm 20\%$
HOLDING TORQUE (N-m)[lb-in]	1.20[10.63]
STEP ANGLE (°)	1.80
STEP ACCURACY (NON-ACCUM)	$\pm 5.00\%$
ROTOR INERTIA (g-cm ²)	300.00
WEIGHT (kg)[lb]	—
TEMPERATURE RISE	MAX 80°C (MOTOR STANDSTILL FOR 2 PHASE ENERGIZED)
AMBIENT TEMPERATURE	-10°C ~ 50°C (14°F ~ 122°F)
INSULATION RESISTANCE	100 M Ohm Min (UNDER NORMAL TEMPERATURE AND HUMIDITY)
INSULATION CLASS B	130°C (266°F)
DIELECTRIC STRENGTH	500 VAC for 1 min (BETWEEN THE MOTOR COILS AND THE MOTOR CASE)
AMBIENT HUMIDITY	MAX 85%(NO CONDENSATION)

Tabla 4 – Tipo de Conexiones (TYPE OF CONNECTION)

TYPE OF CONNECTION		MOTOR	
PIN NO	BIPOLAR	LEADS	WINDING
1	A+ —	BLK (Black)	A+
2	A- —	GRN (Green)	A-
3	B+ —	RED (Red)	B+
4	B- —	BLU (Blue)	B-

